

# Rust & Python

*Le menu parfait ?*



**Mise en bouche**

**Pourquoi appeler du Rust depuis  
Python ?**

# \$ whoami

## \$ whoami

**Igor Laborie** *Founding backend engineer*

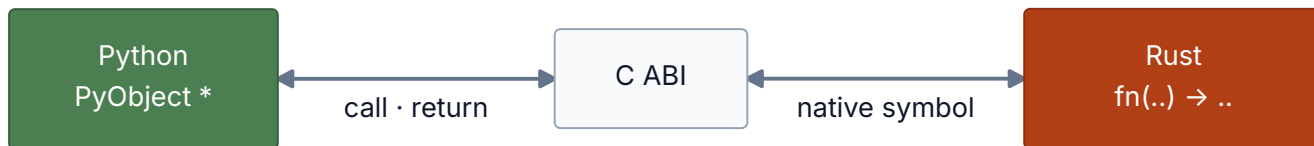
😍 Rust 🤔 Python

# Le constat

- 🐍 Python peut appeler du C avec `ctypes`, `ctypes`, `ctypes`, C-API
- 🦀 Rust peut produire du C

```
1 #[repr(C)]                // Représentation mémoire C
2 pub struct Point { x: f64, y: f64 }
3
4 #[unsafe(no_mangle)]      // Le nom est préservé `norm`, sinon `_ZN4norm...`
5 pub extern "C" fn norm(p: *const Point) → f64 { /* ... */ }
```

 Rust



Techniquement, ça marche C'est quoi le problème ?

# Interopérabilité

| Approche      | Pour               | Contre                 |
|---------------|--------------------|------------------------|
| ctypes        | pas de compilation | lent, aucune sécurité  |
| cffi          | mode ABI ou API    | une dépendance de plus |
| C-API CPython | perf. maximale     | à faire manuellement   |
| <b>PyO3</b>   | Rust + perf C-API  | compiler du Rust       |

PyO3: conversions de types, exceptions, docstrings, signature Maturin: build & wheels

**Bidirectionnel** : Python → Rust, et Rust → Python

# Quel intérêt ?

Rust offers **power and precision** to go beyond Python's limits.

- David Hewitt, mainteneur de PyO3

Fiabilité, performance , sûreté mémoire, concurrence

Polars, DataFusion, pydantic-core, tokenizers, orjson, ..

La stack Python qui va vite est déjà écrite en **Rust**

# Ce que PyO3 doit réconcilier

|             | Python                       | Rust                                                                       |
|-------------|------------------------------|----------------------------------------------------------------------------|
| Typage      | dynamique, à l'exécution     | statique, à la compilation                                                 |
| Objets      | héritage, monkey-patching    | structs + traits                                                           |
| Collections | <code>dict</code> hétérogène | <code>HashMap&lt;K, V&gt;</code> K et V fixer à la compilation             |
| Erreurs     | exceptions                   | <code>Result&lt;T, E&gt;</code>                                            |
| Mémoire     | refcount + GC                | Ownership, pas de GC                                                       |
| Threads     | GIL                          | protection enver les data race avec <code>Send</code> et <code>Sync</code> |

Deux modèles très différents PyO3 fait la traduction

**La mise en place**

**PyO3 + Maturin, et on écrit du code**

# Deux fichiers, et c'est tout

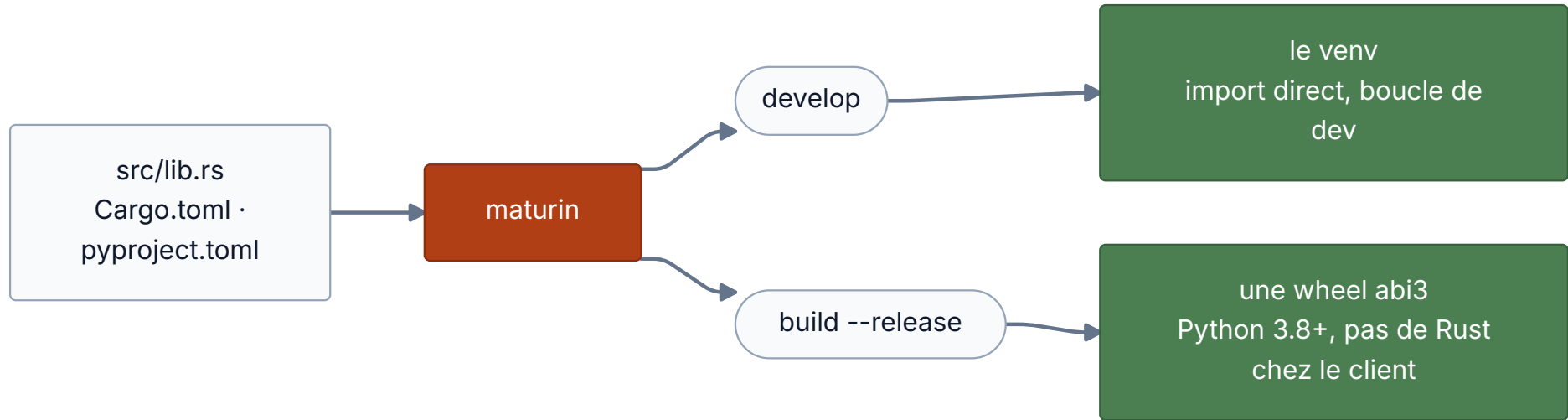
```
1 # Cargo.toml
2 [lib]
3 crate-type = ["cdylib"] # C dynamic library (.so, .dll)
4
5 [dependencies]
6 pyo3 = "0.29"
```

toml

```
1 # pyproject.toml
2 [build-system]
3 requires = ["maturin ≥ 1.9, < 2.0"]
4 build-backend = "maturin"
```

toml

# De la source à la wheel



```
1 $ maturin develop          # compile + installe dans le venv
2 $ maturin build --release  # produit une wheel
```

Shell

# Le résultat

```
1  /// Markdown module
2  #[pyo3::pymodule]
3  mod md {
4      use pyo3::prelude::*;
5      // ...
6
7      /// Render markdown content into HTML
8      #[pyfunction(signature = (md:"str", gfm = false))]
9      pub fn to_html(md: &str, gfm: bool) → PyResult<String> {
10         let mut options = Options::default();
11         let mut plugins = Plugins::default();
12         // ...
13         let result = comrak::markdown_to_html_with_plugins(md, &options, &plugins);
14         Ok(result)
15     }
16 }
```



# Dans l'autre sens

Un binaire **Rust** qui embarque un interpréteur **Python**.

```
1 fn main() → PyResult<()> {
2     Python::attach(|py| {
3         let math = py.import("math"?;
4         let pi: f64 = math.getattr("pi")?.extract()?;
5         println!("π = {pi}");
6
7         py.run(c"print('Bonjour depuis Python')", None, None)
8     })
9 }
```

 Rust

```
1 pyo3 = { version = "0.29", features = ["auto-initialize"] }
```

 toml

# Et si on réécrivait Python ?

Monty : un interpréteur Python écrit en Rust, pour exécuter du code non fiable

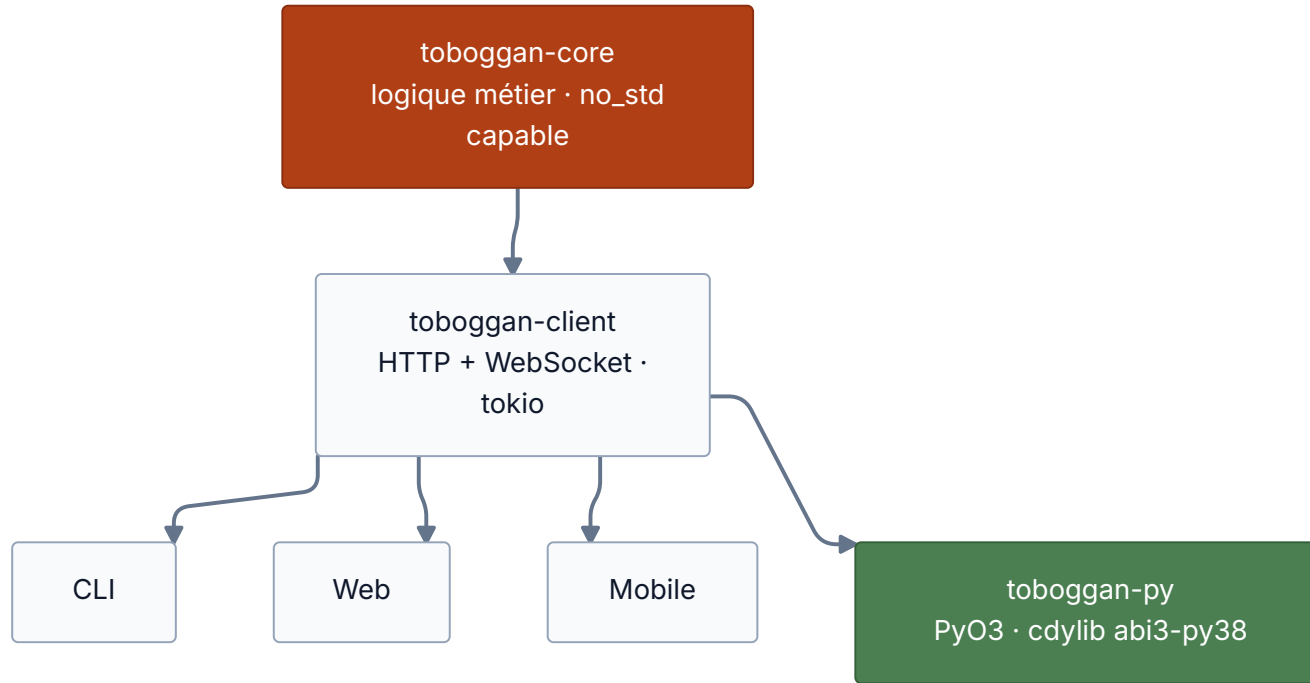
- **PyO3** embarque CPython : tout Python, tout l'écosystème
- **Monty** le remplace : un sous-ensemble de Python, isolé de l'hôte

En Python avec `pydantic-monty...` écrit avec PyO3

**Le coup de feu**

**Deux projets, en production**

# Un cas réel : toboggan-py



Un même client, exposé en CLI, Web, Mobile et Python



## Important

L'API Rust est *async* L'API Python est *sync* (par choix)

# Rust async, Python sync

```
1 #[pyclass]
2 pub struct Toboggan {
3     rt: Runtime,           // le runtime tokio
4     api: TobogganApi,
5     state: Arc<RwLock<TState>>, // l'état
6 }
```

 Rust

## Warning

En appelant `block_on` dans chaque `#[pymethods]`, pas besoin de `await` en Python

# Le piège du GIL

```
1 // `block_on` verrouille le GIL pendant les I/O
2 let notif = self.rt.block_on(self.api.command(cmd));
```

 Rust

```
1 // `detach` relache le lock pendant les I/O
2 let notif = py.detach(|| self.rt.block_on(self.api.command(cmd)));
```

 Rust

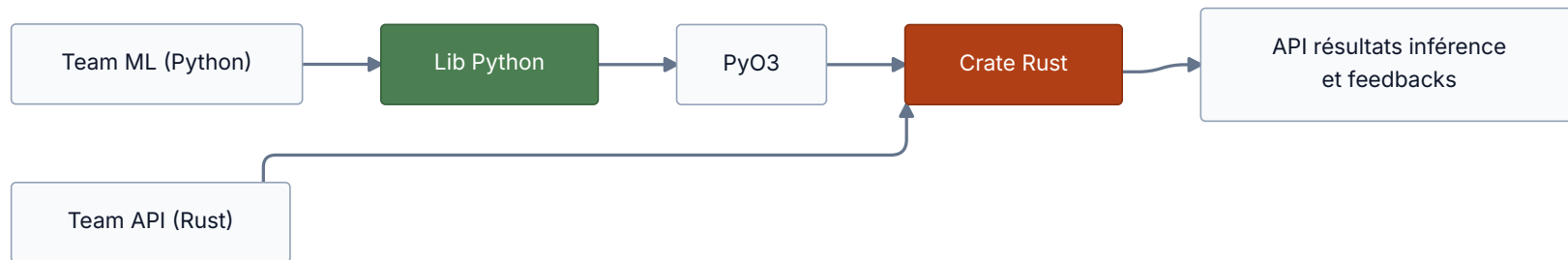
```
1 // L'autre choix avec pyo3-async-runtimes
2 // En Python: await fetch(url)
3 #[pyfunction]
4 fn fetch(py: Python<'_, url: String> → PyResult<Bound<'_, PyAny>> {
5     pyo3_async_runtimes::tokio::future_into_py(py, async move {
6         let rsp = request::get(&url).await.map_err(to_py)?;
7         rsp.text().await.map_err(to_py)
8     })
9 }
```

 Rust

[pyo3.rs/v0.29.3/async-await](https://pyo3.rs/v0.29.3/async-await)

# Chez Wefox AI: une API client, deux équipes

Chez Wefox AI: micro service pour stocker les résultats d'inférence et leurs feedback



Implémentation partagée, un contrat adapté à chaque équipe, on masque le détail d'implémentation

# Télémétrie: comment propager la trace?



| Ce qui ne traverse pas        | Sens          | Le pont               |
|-------------------------------|---------------|-----------------------|
| Les logs <code>tracing</code> | Rust → Python | <code>pyo3-log</code> |
| Le contexte de trace          | Python → Rust | W3C TraceContext      |

# Les logs passent

```
1 // tracing = { version = "0.1", features = ["log-always"] }
2 #[pymodule]
3 mod rusty_otel {
4     #[pymodule_export]
5     use super::render;
6
7     #[pymodule_init]
8     fn init(m: &Bound<'_, PyModule>) → PyResult<()> {
9         pyo3_log::init();    // log → logging Python
10        telemetry::init();    // subscriber tracing + exporter OTel
11        Ok(())
12    }
13 }
```

 Rust

```
1 import rusty_otel    # appel de init
```

 Python

```
1 INFO app          [trace=6a79...5545] appel du moteur Rust
2 INFO rusty_otel    [trace=6a79...5545] rendu de 53 octets de markdown
```

 Text

# Le contexte aussi

```
1 // Récupère le contexte OTel de Python, et le passe en Rust
2 fn python_context(py: Python<'>) → PyResult<Context> {
3     let carrier = PyDict::new(py);
4     py.import("opentelemetry.propagate")?
5         .call_method1("inject", (&carrier,))?; // {"traceparent": "00-6a79...-01"}
6
7     let carrier = carrier.extract::<HashMap<String, String>>()?;
8     let context = global::get_text_map_propagator(|prop| prop.extract(&carrier));
9     Ok(context)
10 }
```

 Rust

```
1 let span = tracing::info_span!("render", bytes = input.len());
2 let parent = python_context(py).unwrap_or_default();
3 if let Err(err) = span.set_parent(parent) {
4     tracing::warn!("contexte Python non rattaché : {err}");
5 }
6 // Tip: utiliser des macro_rules pour simplifier ce code
```

 Rust

[pyo3.rs/v0.29.3/ecosystem/tracing](https://pyo3.rs/v0.29.3/ecosystem/tracing)

**L'addition**

**Ce que ça coûte, ce que ça rapporte**

# Ce qui coince

- Pas de génériques dans les classes: `#[pyclass] struct Container<T>`
- Pas de lifetime dans les classes: `#[pyclass] struct Container<'a>`
- Pas d'interop traits vers protocoles, par ex. `Iterator` ne devient pas itérable sans écrire manuellement les `__iter__` & `__next__`
- La génération des `.pyi` n'est pas automatique, Python typing hints, #5137
- Ne pas bloquer le GIL

# Tips

- Réduire la surface d'interop
- Batcher, un appel pour 1 M d'éléments, pas 1 M d'appels
- `maturin-import-hook` ou un watcher comme `bacon`
- `py.detach(...)` sur les sections CPU-bound et les I/O bloquantes si on `block_on`
- Tester des deux côtés avec `pytest` et `cargo test`
- `py-spy`: le flamegraph peut voir les frames Rust
- Faire attention aux mesures de performance: `--release`

# Quand ça vaut le coup

1. **Optimisation**: garder l'API Python, remplacer les modules à optimiser par du Rust. Par exemple Polars, pydantic-core, ...
2. **API polyglotte**: un noyau Rust, exposé partout Par exemple OpenDAL, xberg, ...
3. **Prototyper en Python puis migrer en Rust**: itérer +vite, faire performant quand on est stable
4. **Sandboxing, plugins**: controller du code Python non-fiable. Mais est-ce que WASM, ou les microVM ne sont pas mieux pour ça ?

# Références

## Merci !

### Des questions ?

---

- **Outils** - [pyo3.rs](#) · [maturin.rs](#) · écosystème PyO3 · [pyo3-async-runtimes](#) · [rust-numpy](#) · [py-spy](#) · [maturin-import-hook](#)
- **Les talks de David Hewitt**, mainteneur PyO3 - [PyO3 in depth](#) · [Free-threaded Python](#) · [5 years of Rust in Python](#) · [ITW - From Python to Rust and Back Again](#)
- **Le code de cette conf** - [github.com/ilaborie/rust-python-talk](#) slides